

EXP -1 :- Basic Logic Gates Using Dataflow Modeling

Aim

To design and verify basic logic gates (AND, OR, NOT, NAND, NOR, XOR, XNOR, BUFFER) in Verilog using dataflow modeling.

Code

```
`timescale 1ns/1ps

module logic_gates_dataflow (
    input wire A,
    input wire B,
    output wire AND_GATE,
    output wire OR_GATE,
    output wire NOT_GATE,
    output wire NAND_GATE,
    output wire NOR_GATE,
    output wire XOR_GATE,
    output wire XNOR_GATE,
    output wire BUFFER_GATE
);

assign AND_GATE    = A & B;
assign OR_GATE     = A | B;
assign NOT_GATE    = ~A;
assign NAND_GATE   = ~(A & B);
assign NOR_GATE    = ~(A | B);
assign XOR_GATE    = A ^ B;
assign XNOR_GATE   = ~(A ^ B);

endmodule
```

Truth Table / Observation

A	B	AND_GATE	OR_GATE	NOT_GATE	NAND_GATE	NOR_GATE	XOR_GATE	XNOR_GATE
0	0	0	0	1	1	1	0	1
0	1	0	1	1	1	0	1	0
1	0	0	1	0	1	0	1	0
1	1	1	1	0	0	0	0	1

Observation: All 4 input combinations were exercised by the testbench and each gate matched its expected Boolean truth table. The NOT_GATE depends only on input A. Note: the module declares a BUFFER_GATE output, but the RTL provides no assign driving it, so that output is left undriven in the delivered code.

Conclusion

The eight basic logic gates were successfully implemented in Verilog using dataflow modeling with continuous assign statements.